T3: Ciclo di vita del SW

Il concetto di “ciclo di vita”

Partiamo da: Concezione -> Sviluppo -> Utilizzo -> Ritiro ma nel corso ci occuperemo principalmente su concezione -> sviluppo.

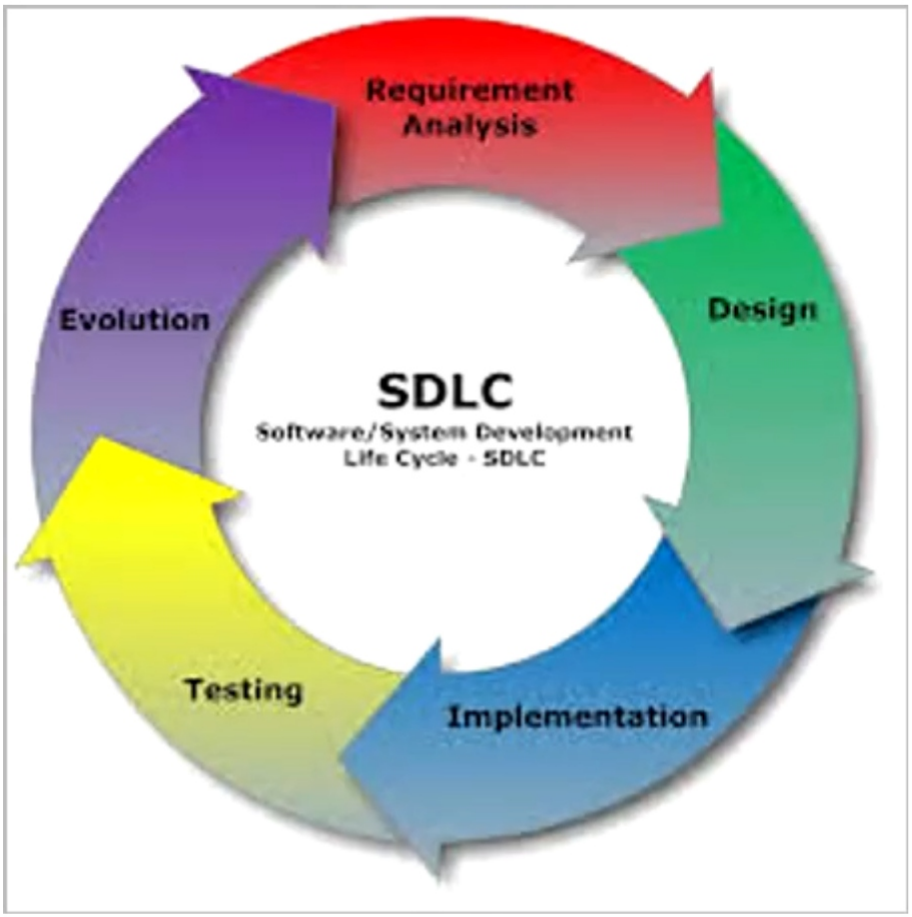
La transizione fra questi stati avviene tramite l’esecuzione di attività di processi di ciclo di vita, che prevedono specifiche responsabilità da cui definiamo dei **ruoli** in un team di progetto.

Lo stazionamento in uno stato/transizione si dice **fase**: “fase” designa un segmento temporale contiguo con caratteristiche coerenti;

Aderire ad un modello comporta vincoli su pianificazione e gestione del progetto: la scelta del modello di ciclo di vita influenza la selezione del *way of working* e dei suoi strumenti di supporto;

Modelli di ciclo di vita

Esistono molteplici cicli di vita che differiscono per transizioni tra stati e regole di attivazione, l’esempio che segue mostra un ciclo di vita che non prevede “Ritiro”:



Come vi sono modelli di ciclo di vita, vi sono anche **modelli di sviluppo**;

Cosa significa “Modello” ?

Definiamo **Modello**: un insieme di specifiche che descrivono un fenomeno di interesse (astratto/concreto) in modo oggettivo, non dipendente dall’osservatore e dimostrato corretto empiricamente o per teorema;

I modelli aiutano a studiare, comrpendere, misurare, trasformare l’oggetto di interesse:

- il modello specifica *cosa* esso sia;
- l'architettura interna (design) specifica *come* esso funzioni;
- l'analisi specifica *perchè* fa quel che fa e come lo fa;

Evoluzione dei modelli di sviluppo

Il non-modello per eccellenza è il *code-n'-fix* delle origini (o cowboy coding) che portava con se un insieme di attività senza organizzazione preordinata;

Questo indusse alla nascita della SWE e di modelli organizzati:

- **Cascata**: rigide fasi sequenziali;
- **Incrementale**: relizzazione in più passi;
- **Evolutivo**: ripetute iterazione che rilasciano diverse varianti di prodotto;
- **A componenti**: orientato al riuso;
- **Agile**: altamente dinamico, fatto di brevi cicli iterativi e incrementali;

Modello sequenziale "a cascata"

Si basa sulla produzione industriale centrato sull'idea di *processi ripetibili*;

Richiede la successione di *fasi rigidamente sequenziali*, non ammette il ritorno a fasi precedenti (se non per eventi eccezionali che riportano all'inizio) da cui ne deriva il nome "a cascata".
Le iterazioni costano troppo e non vengono viste come buon mezzo di mitigazione delle incertezze di sviluppo;

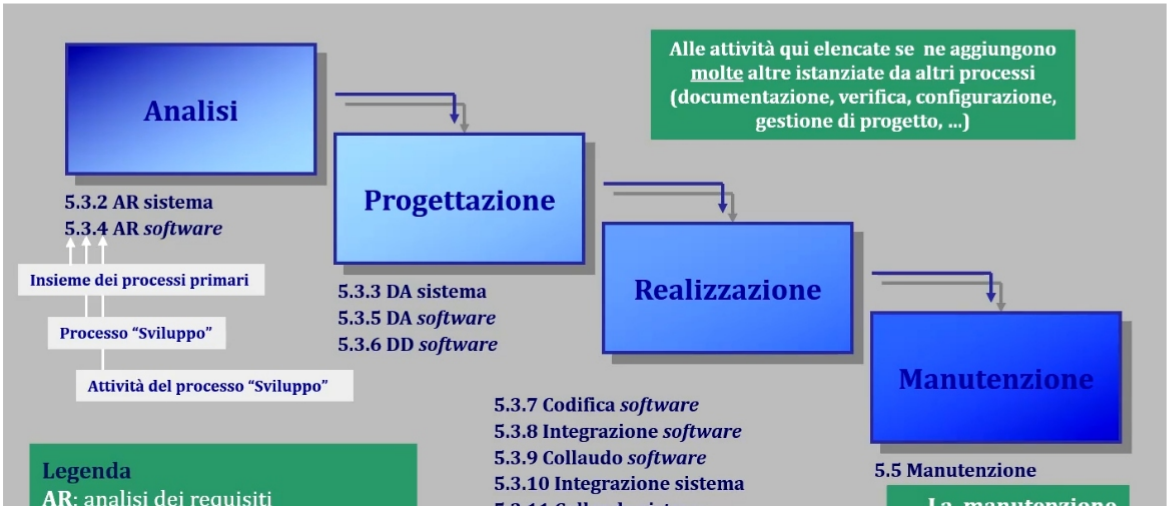
Prodotti: principalmente *documenti*, fino a poi includere il SW;

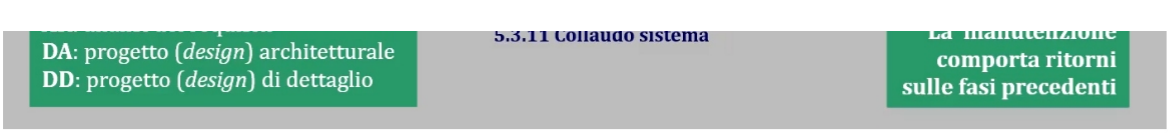
L'ingresso in uno stato è condizionato da *pre-condizioni* (gate) che sono le *post-condizioni* delle corrispondenti transizioni;
E' diviso in fasi distinte e non sovrapposte nel tempo: modello adatto allo sviluppo di sistemi complessi anche sul piano organizzativo.

Ogni fase (stato o transizione) viene definita in termini di:

- Attività previste e prodotti atteso in ingresso ed in uscita;
- Contenuti e struttura dei documenti;
- Responsabilità e ruoli coinvolti;
- Scadenze di consegna dei prodotti;

Entrare, uscire, stazionare in una fase comporta lo svolgimento di determinate azione realizzate come attività erogate da specifici processi.





Critiche:

- Eccessiva rigidità;
- Stretta sequenzialità: nesun parallelismo e nessun ritorno;
- Non ammette modifiche nei requisiti in corso d'opera;
- Visione rigida (burocratica) e poco realistica del progetto.

Correttivi:

- Prototipi di tipo "usa-e-getta":
  - per capire meglio i requisiti o le soluzioni;
  - strettamente all'interno di singole fasi;
- Cascata con ritorni: ogni ciclo di ritorno raggrupa sotto-sequenze di fasi;

Ritorni: iterazione o incremento?

Puo spesso capire di trovare *stakeholders* che non hanno le idee chiare, per aiutarli possiamo procedere:

- Iterando (potenzialmente distruttive): per problemi particolarmente complessi che richiedono di procedere a tentoni;
- Incrementando:
  - posticipare l'integrazione delle parti del sistema è rischioso;
  - se le cose vanno male restiamo con nulla in mano (*big-bang integration*);
  - meglio l'**integrazione continua**;

In generale iterazione ed incremento coincidono quando la sostituizione raffina ma non ha impatto sul resto.

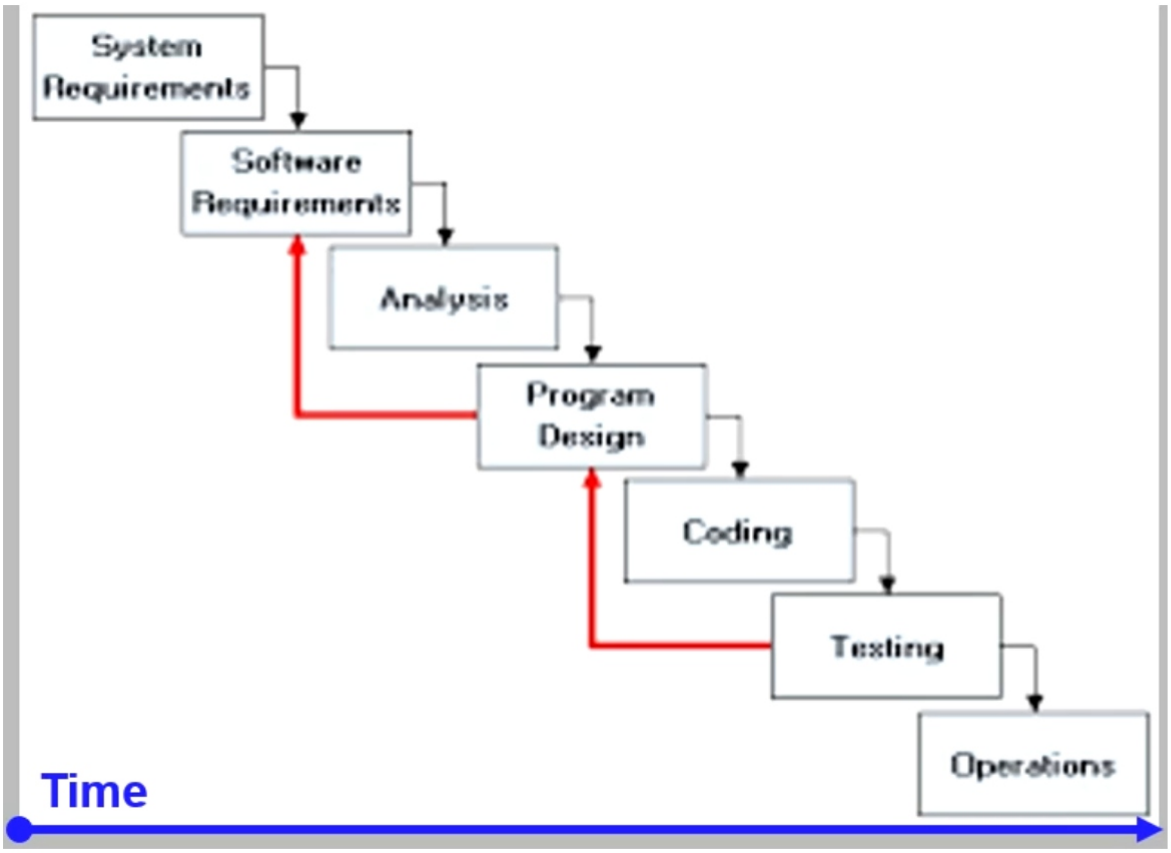
Vantaggi dei modelli incrementali

1. Possono produrre valore ad ogni incremento:
   - un insieme di funzionalità diventa presto disponibile;
   - i primi incrementi possono essere frutto di prototipazione aiutando a fissare meglio i requisiti per gli incrementi successivi;
2. Ogni incremento riduce il rischio di fallimento (non lo azzera!);
3. Le funzionalità fondamentali vanno sviluppate prima:
   - cosi che esse siano più frequentemente verificate e diventando via via più stabili;

Vantaggi dei modelli iterativi

1. Applicabili a qualunque modello di sviluppo ma comportando forte potenziale distruttivo;
2. Consentono maggior capacità di adattamento:
   - insorgere di problemi, cambio di requisiti, collasso tecnologico;
3. Comportano il rischio di non convergenza (impossibile uscirne);
4. Soluzione generale:
   - decomporre la realizzazione del sistema;
   - identificare e lavorare *prima* sulle parti più critiche (quelle più complesse o quelle con i requisiti più incerti);

◦ limatando superiormente il numero di iterazioni;



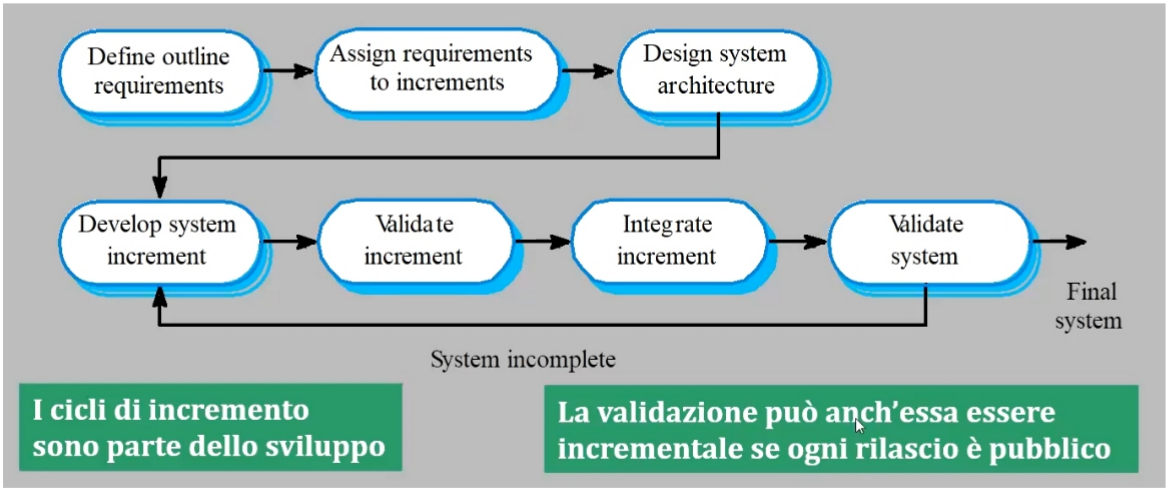
N.B. Ogni iterazione comporta un ritorno nella direzione *opposta* all'avanzamento del tempo;

Modelli incrementali

Prevede rilasci multipli e successivi, ciascuno realizza un incremento di funzionalità;

I **requisiti** sono classificati e trattati in base alla loro importanza strategica: - i primi incrementi puntano a soddisfare i requisiti più importanti sul piano strategico; - cosi i requisiti importanti diventano presto chiari e stabili, quindi più facilmente soddisfacibili; - quelli meno importanti hanno invece più tempo per stabilizzarsi e armonizzarsi con lo stato del sistema;

In generale lo schema è quello che segue:



L'analisi dei requisiti e progettazione architetturale vengono svolti una volta sola:

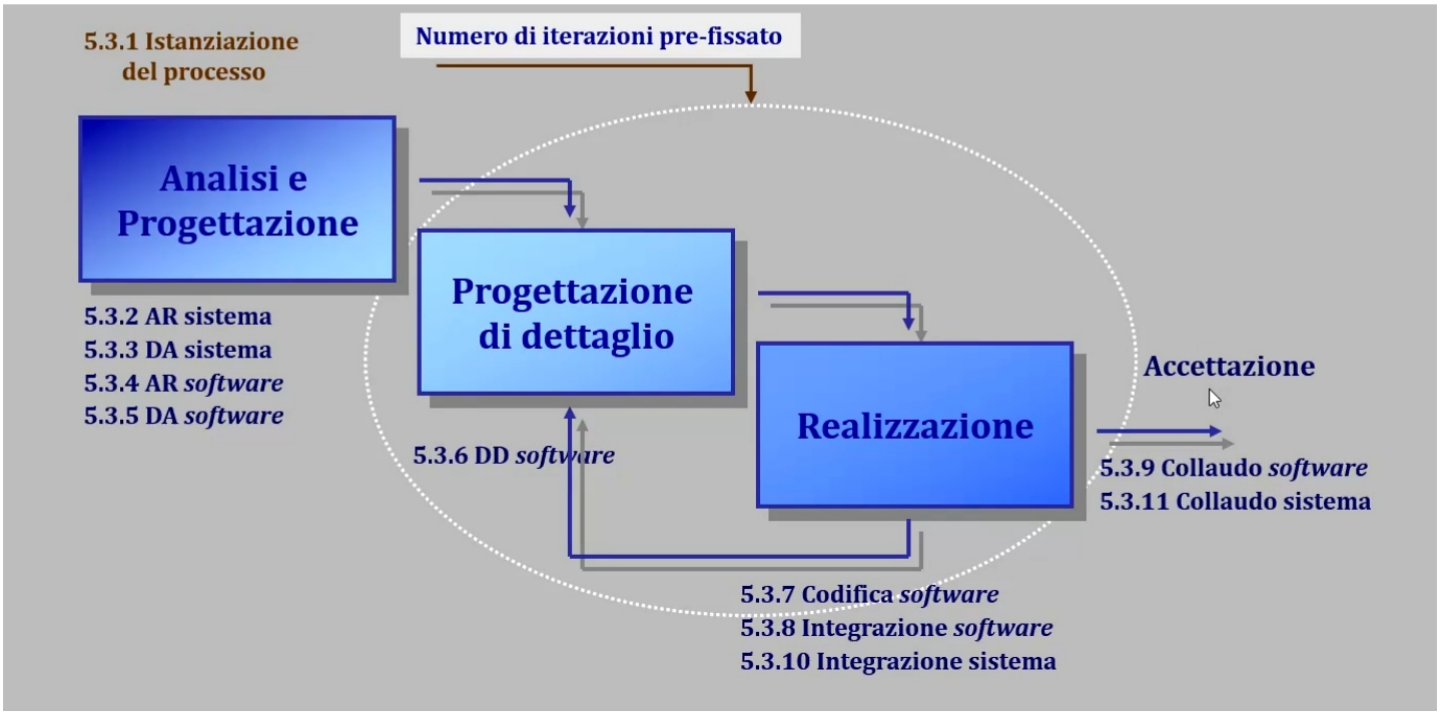
- per stabilizzare presto i requisiti principali;

- per stabilizzare presto l'**architettura** complessiva del sistema;
- per decidere preventiamente il numero di incrementi;

La realizzazione è incrementale:

- l'analisi dei requisiti puo' essere raffinata;
- il completamento dei primi incrementi deve rendere disponibili le principali funzionalità;

Secondo ISO 12207 lo schema è il seguente:



Modello evolutivo

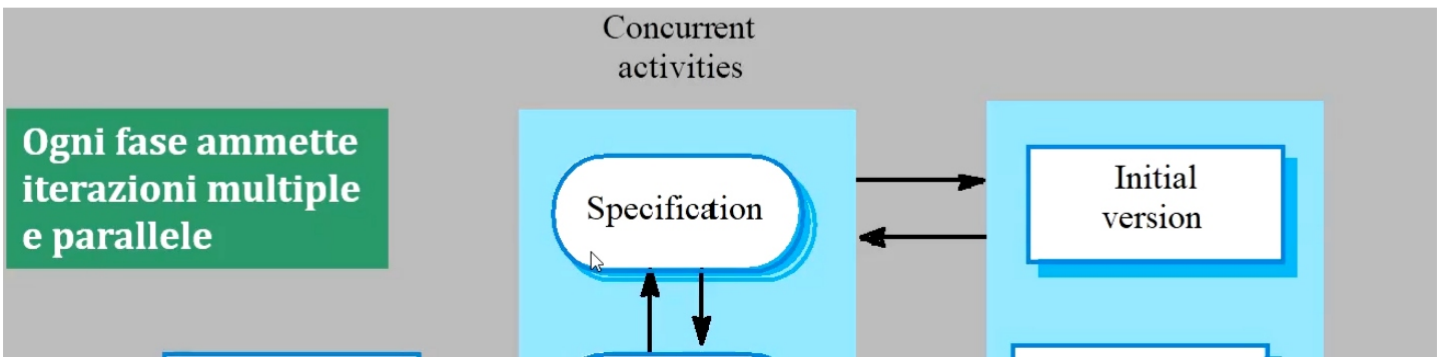
Aiuta a rispondere a bisogni non inizialmente preventivabili, puo' richiedere il rilascio ed il mantenimento di più versioni esterne attive in parallelo e comporta il riattraversamento di più stati di ciclo di vita.

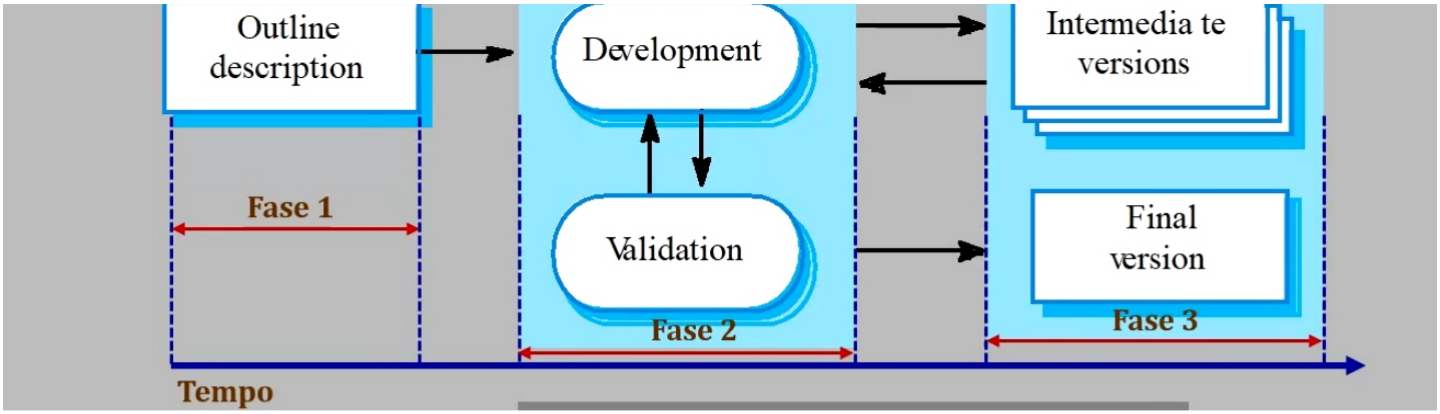
Spesso viene chiamato anche "train model", intendendo che vengono mantenuti in parallelo diversi canali per le diverse versioni.

Analisi preliminare:

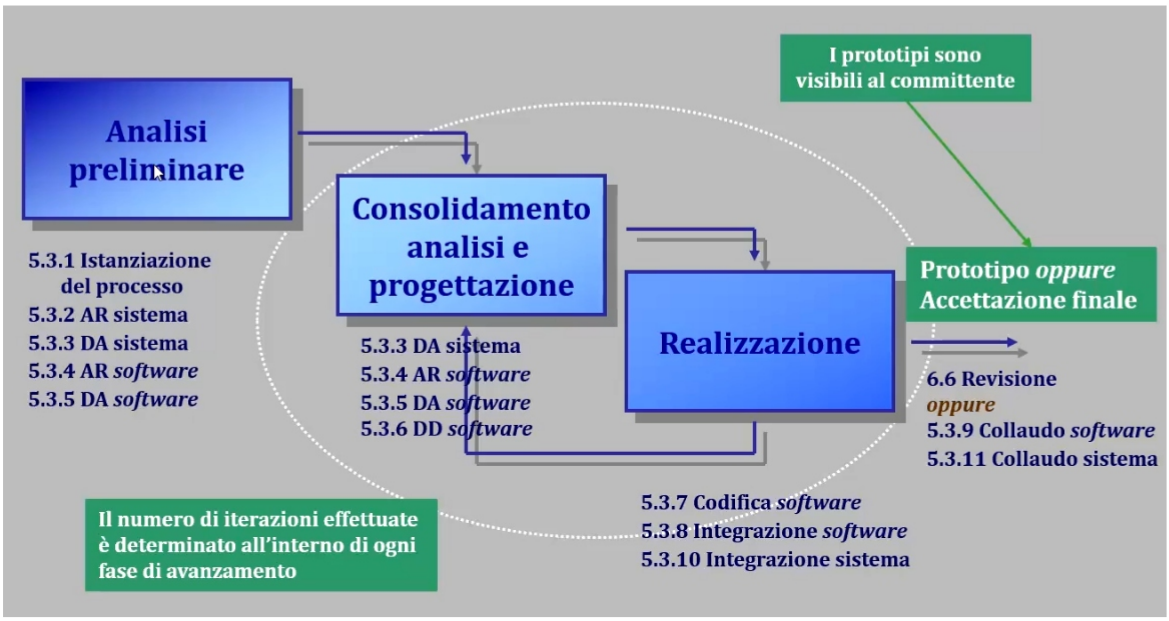
- identificare i requisiti fondamentali;
- definire l'architettura di base, massimamente modulare per facilitare evoluzione futura;
- pianificare i passi di analisi e realizzazione successivi, anche sovrapponendoli fra loro.

Analisi e realizzaizone di una singola evoluzione raffinano ed estendono lo stato precedente, mentre ogni evoluzione rilascia versioni indipendenti (fintanto che necessario).





Secondo ISO 12207 lo schema è il seguente:



Modello a componenti

Molto di quello che ci serve fare è già stato fatto e molto di quello che faremo ci potrà servire ancora:

- analisi dei requisiti guidata dalla possibilità di riuso di quanto già esista;
- realizzazione che cerca di favorire riuso futuro;



Metodi agili

Nascono alla fine degli anni '90 in contrasto all'eccessiva rigità dei modelli allora presenti.

Si basano su quattro prinicipi fondanti:

1. Individuals and interactions over processes and tools:
    - l'eccessiva rigidità ostacola l'emergere del valore;
2. Working software over comprehensive documentation:
    - la documentazione non sempre corrisponde a SW funzionante;
3. Customer collaboration over contract negotiation:
    - l'interazione con gli stakeholder va incentivata e non ingessata;
4. Responding to change over following a plan:
    - la capacità di adattamento al cambiare delle situazioni è importante;

Dato un modello posso avere un medoto che lo attua, i metodi agili ruotano attorno ad una "user story" (il mondo visto dall'utente che ancora non esiste): una funzionalità significativa che l'utente vuole realizzare con il SW richiesto.

Ogni "user story" è definita da:

- un documento di descrizione del problema individuato;
- la **minuta** delle conversazioni con gli *stakeholders* effettuate per discutere e comprendere il problema;
- la strategia da usare per confermare che il SW realizzato soddisfi gli obiettivi di quel problema;

Il modello incrementale è il paradiso dell'agile.

Assunti base

1. Suddividere il lavoro in piccoli incrementi a valore aggiunto, magari anche sviluppabili indipendentemente;
2. Sviluppare ciascun incremento in sequenza continua dall'analisi all'integrazione;

Obiettivi strategici

1. Poter sempre dimostrare al cliente quanto è stato fatto;
2. Verificare l'avanzamento tramite progresso reale;
3. Dare agli sviluppatori la soddisfazione del risultato

Un buon esempio è lo **Scrum** (organizzazione dietro caos apparente), **Kanban** (just-in-time), **Scrumban**.

Contro-argomentazioni

SW privo di docunmentaizone produce costo, non valore perchè commentare il codice *non* basta ma serve motivare le scelte realizzative.

Senza un piano non si possono valutare rischi ed avanzamenti, la sola misurazione di consuntivo *non* puo' bastare.

Cambiare si puo' ma con consapevolezza del rapporto costo/benefici.

Metodo Scrum (Agile)

In generale questo è il framework scrum:





- **Product Backlog**
  Requisiti e funzionalità del prodotto
- **Sprint Backlog**
  Insieme di storie del prossimo sprint
- **Sprint**
  Fase operativa di sviluppo
  Durata media 2 - 4 settimane
  Prodotto potenzialmente vendibile

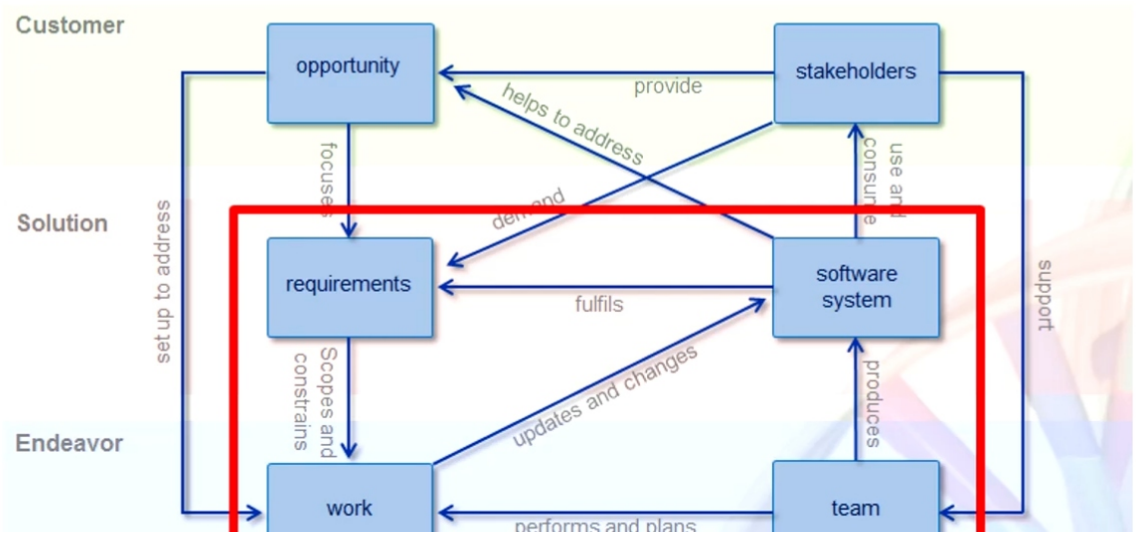
Più in particolare, uno **sprint** è articolato come segue:

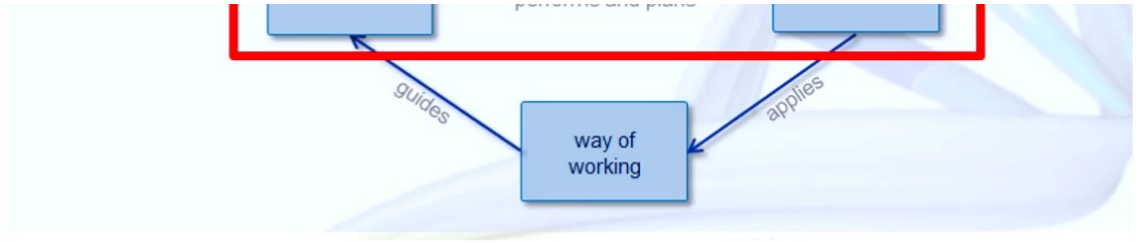


- **Sprint Planning**
  Pianificazione dello sprint
- **Sprint Review**
  Controllo prodotti dello sprint
- **Daily Scrum**
  Controllo giornaliero avanzamento
- **Sprint Retrospective**
  Controllo qualità sullo sprint

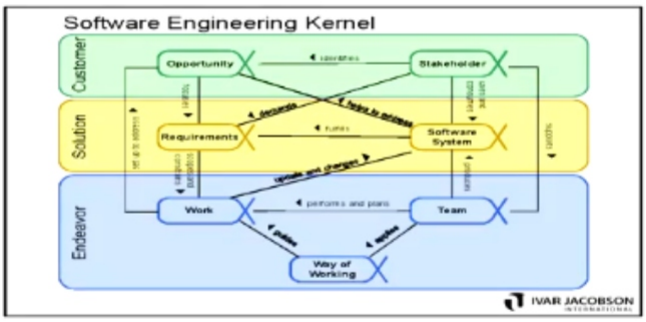
Ciclo di vita secondo SEMAT

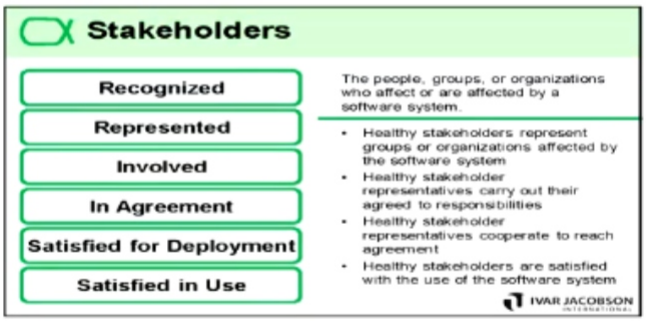
SEMAT, per ogni elemento di quello che definice Software Engineering Kernel:

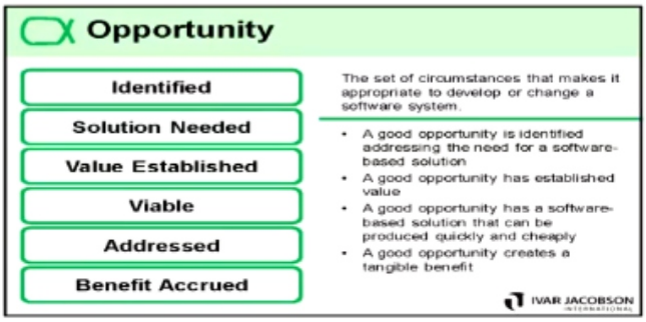


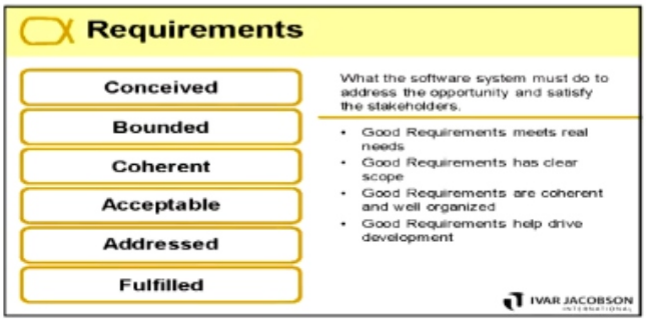


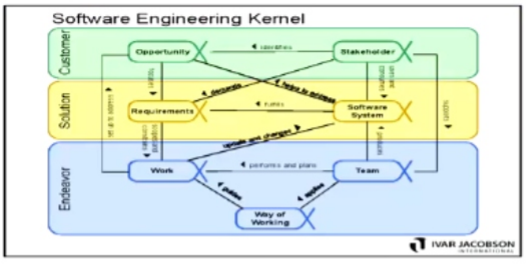
definisce un “termometro di avanzamento” (*scorecard*) composto da 6 stati che il prodotto SW attraversa e ci aiuta a capire a che punto siamo e a che punto dovremmo essere.

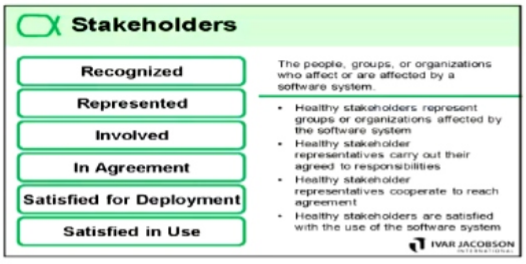


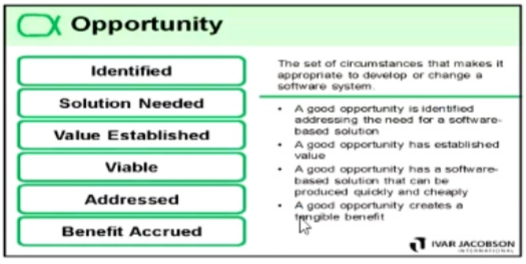


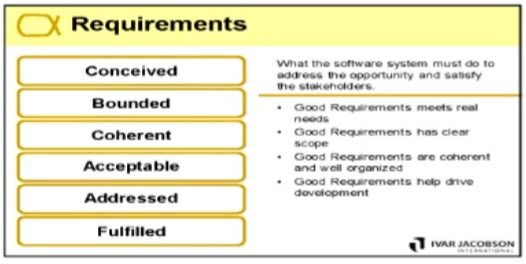






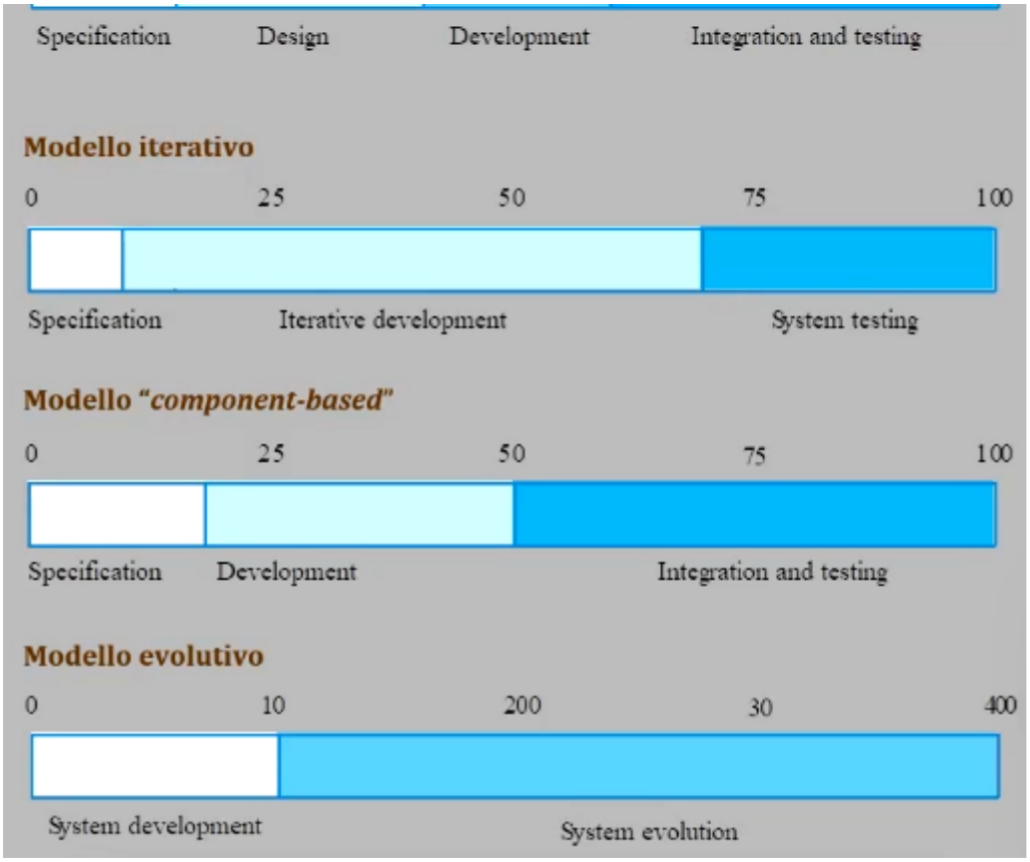






Ripartizioni dei costi nei modelli




Specification
Design
Development
Integration and testing
Modello iterativo
0
25
50
75
100
Specification
Iterative development
System testing
Modello "component-based"
0
25
50
75
100
Specification
Development
Integration and testing
Modello evolutivo
0
10
200
30
400
System development
System evolution